

datalab 报告

姓名：凌小雅 学号：2025200717

test 截图：

```
● yuanshan@LAPTOP-VBIA808D:~/datalab2026$ python3 test.py
rm -f *.o btest fshow ishow *~ yacctab.py lextab.py result.txt .autograder_result
gcc -O0 -Wall -std=gnu99 -lm -o btest bits.c btest.c decl.c tests.c
gcc -O0 -Wall -std=gnu99 -o fshow fshow.c
gcc -O0 -Wall -std=gnu99 -o ishow ishow.c
Make success.
bitAnd          1/1:    PASS
bitXor          1/1:    PASS
samesign        2/2:    PASS
logtwo          4/4:    PASS
byteSwap        4/4:    PASS
reverse         3/3:    PASS
logicalShift    3/3:    PASS
leftBitCount    4/4:    PASS
float_i2f       4/4:    PASS
floatScale2     4/4:    PASS
float64_f2i     3/3:    PASS
floatPower2     4/4:    PASS
Total points: 37
```

解题报告

亮点

1. logtwo
2. leftBitCount
3. float_i2f

logtwo

```
int logtwo(int v) {
    int res = 0;

    int shift;

    shift = (v > 0xFFFF) << 4;

    res = res | shift;

    v = v >> shift;

    shift = (v > 0xFF) << 3;

    res = res | shift;

    v = v >> shift;
```

```

    shift = (v > 0xF) << 2;

    res = res | shift;

    v = v >> shift;

    shift = (v > 0x3) << 1;

    res = res | shift;

    v = v >> shift;

    shift = v > 1;

    res = res | shift;

    return res;

}

```

讲解题目思路 在不能使用加法的情况下，利用从高位开始比较，通过移位找 1 的办法，寻找最高位 1 所在位置，并使用 “|” 代替加法功能，最终参考分治思路完成题目。

leftBitCount

```

int leftBitCount(int x) {

    int res = 0;

    int shift = 0;

    shift = (!~(x >> 16)) << 4;

    res += shift;

    x = x << shift;

    shift = (!~(x >> 24)) << 3;

    res += shift;

    x = x << shift;

    //以此类推看高 4 位 高 2 位和高 1 位

    shift = (!~(x >> 31));

    res += shift;

    x = x << shift;
}

```

```

    shift = (!~(x>>31));

    res += shift;

    //最后移位要重复进行一次防止漏掉最后一位
}

```

讲解题目思路 在 `logtwo` 的基础上，使用 `shift = (!~(x>>16))<<4`;代替比较，巧妙避开“>”的使用。

float_i2f

```

unsigned sign, ux, frac;

    int exp = 158;

    sign = x & 0x80000000u;

    ux = x;

    if (sign) ux = ~ux + 1;

    while (!(ux & 0x80000000u)) {

        ux <<= 1;

        exp -= 1;

    }

    //找出 frac 对应位数

    frac = ux >> 8;

    //向偶数取整

    if ((ux & 0xFFu) > 0x80u) frac += 1;

    if ((ux & 0x1FFu) == 0x180u) frac += 1;

    return sign + (exp << 23) + frac;

```

讲解题目思路 分别提取 `sign`、`exp`、`frac`，其中 `exp` 设置为 158 ($31 + 127$) ($(E_{\max} + bias)$)；向偶数取整同样运用合理比较减少操作符使用。

反馈 / 收获 / 感悟 / 总结

花费时间：约 7h 难度：较高 合理：按照课堂讲解顺序，运算符→编码→int→浮点数
有趣：部分算法在 ai 帮助下优化，发现 ai（严格来说是前人）解法堪称天才，使人茅塞顿开。

参考的重要资料

课堂 PPT: bitcount 部分习题以及分治思想 《深入理解计算机系统》第三版 浮点数 IEEE 浮点表示部分。